Results: SkillTree Export and Learner-Surface Handoff

SkillTree export, learner-surface handoff, and generated front matter I

All registered claims are produced by tested kernels; every claim token in the 630 skills resolves, and the bare-number linter reports zero violations across descriptions, assessment prompts, and correct numeric answer options. The scholarship audit similarly resolves every authored citation against the bibliography and source-role matrix. The provenance-rendered graph exports to a SkillTree project of 111 subjects, 630 skills, 1199 learning-path dependencies, 630 quizzes, and 2520 assessment questions, with no unresolved tokens in the output. The source reaches the configured `src/` coverage gate with no mocks; exact current coverage is reported by the test run rather than asserted in prose.

SkillTree export, learner-surface handoff, and generated front matter I

The dashboard delivery surface exposes the same contract. Its status payload cross-checks SkillTree export counts against build stats, tracks the claim ledger and figure artifacts, exposes the manuscript artifact audit, mounts the official learner display when SkillTree is online, and gives each skill a direct handoff into that display's subject and quiz-gated skill pages. This avoids the common false demo in which a polished local UI merely copies course text: the local page can inspect and preview artifacts, but the learning and quiz actions are routed to the seeded SkillTree project.

Prose cannot settle that distinction, so the delivery surface is also recorded directly. fig. 1 shows the landing view reporting exported counts, live SkillTree dependency parity, and artifact presence together. fig. 2 shows SkillTree's own client library rendering the seeded project inside the local page, with its native points, level, progress rings, and ranking; that is the observation the export-lands-in-the-platform reading needs, and no exported JSON file can supply it. fig. 3 shows the generated lesson record for one skill beside its commit-pinned upstream pointers, so the material boundary is visible rather than asserted. fig. 4 shows the captioned prerequisite-flow table that carries the heaviest subject-to-subject relations for readers who cannot use the lane drawing, which is a ranked

summary of that structure rather than a complete substitute for every edge, and fig. 5 shows deployment-mode readiness beside the browser-visible runtime payload, where the admin password and client secret are absent. These five images are recordings rather than renders: unlike every other figure here they cannot be rebuilt from project data, so they are version-controlled, marked in the figure registry as captured evidence, and stamped with the dashboard URL, viewport, live counts, capture timestamp, per-image digest, and full-stack mode of the run that produced them. Being recordings, they are also the one family exempt from the check that every registered figure was emitted by the render pipeline; the exemption is recorded in the registry rather than left implicit, and their integrity rests on the digest and provenance record instead.

The front matter follows the same rule. The cover image is generated from the live graph, claim registry and roadmap metrics, and the page-two named-skill atlas is generated from every authored skill name in the curriculum graph. Both images are checked by publication-asset audits before rendering. The cover uses a square layout so it can serve both PDF title-page and web artifact contexts; the atlas uses a wide page-two layout so the skill-name vocabulary remains readable by showing a reduced weighted-term sample and representative dependency checkpoints rather than a complete printed index. Neither asset is a numbered scientific figure: the dashboard exposes them as cover-category artifacts, while `figure_registry.json` remains reserved for manuscript figures that are declared and cross-referenced in the text. This prevents polished front matter from becoming a false-certification

path outside the artifact contract.

SkillTree export, learner-surface handoff, and generated front matter I

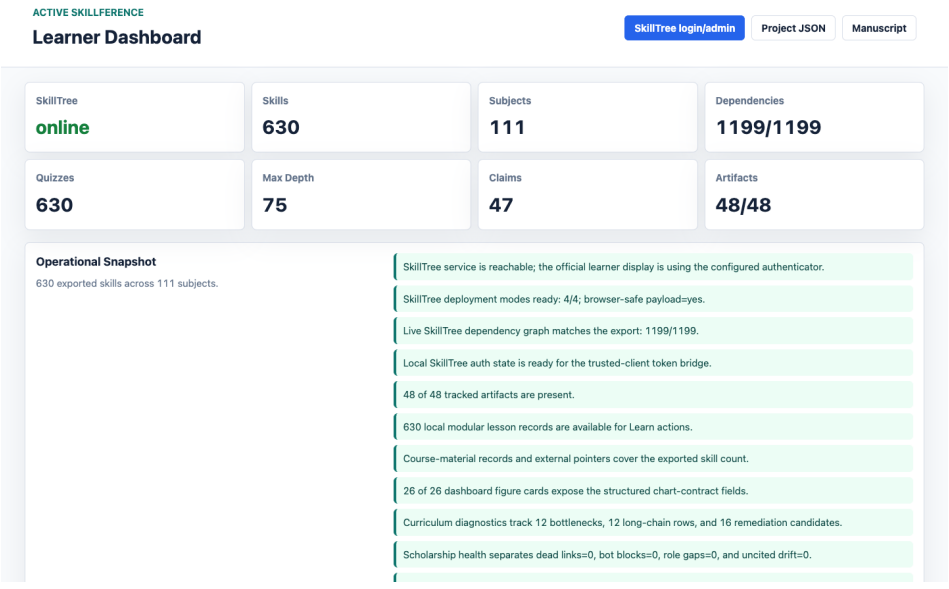


Figure 1: Learner dashboard landing view captured from the running local stack. The metric row reports exported skills, subjects, quizzes, maximum prerequisite depth, registered claims, and artifact presence, while the dependency metric shows live SkillTree parity against the export rather than the exported count alone. The evidence list below restates those machine-checked conditions in words: export counts, live graph parity, local auth readiness, course-material coverage, and figure-contract completeness are surfaced together on one page instead of being scattered across separate reports. The list continues past the fold; the frame shows the first ten of its rows. The artifact tally counts this evidence family among the tracked artifacts, so it is a completeness check on the roster rather than an independent audit of these images.

SkillTree export, learner-surface handoff, and generated front matter II

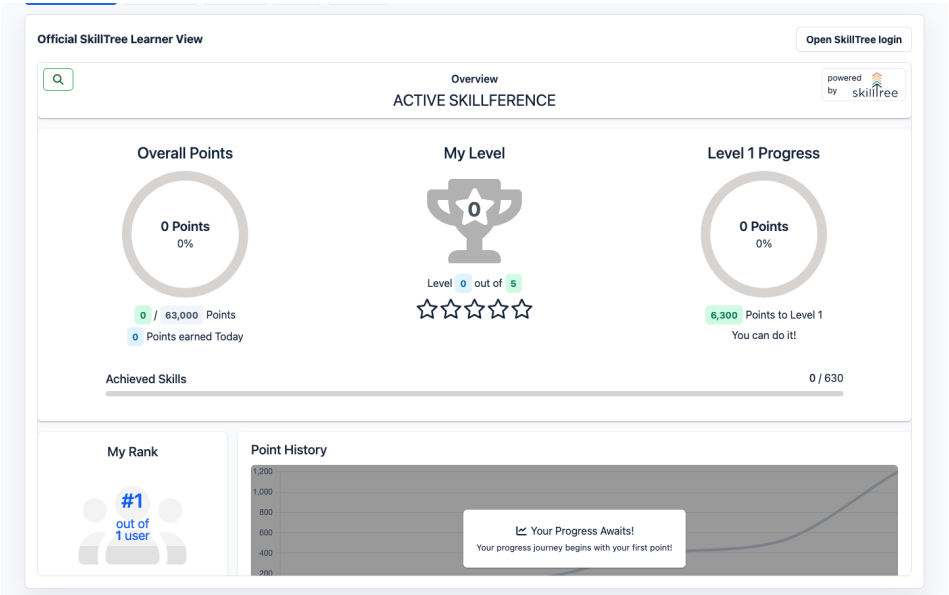


Figure 2: The Official Display tab captured while the embedded SkillTree learner client is mounted against the seeded project. The overview header, overall points, level and level progress rings, the achieved-skills bar reading zero of the full exported skill count, the personal rank card, and the point-history panel are all rendered by SkillTree's own client library inside the project dashboard, which is the demonstration that the export lands in the platform runtime rather than in a project reimplementation of it. Every progress value reads zero and the point-history panel shows its first-point empty state, because the capture uses a freshly seeded local identity that has completed nothing.

SkillTree export, learner-surface handoff, and generated front matter III

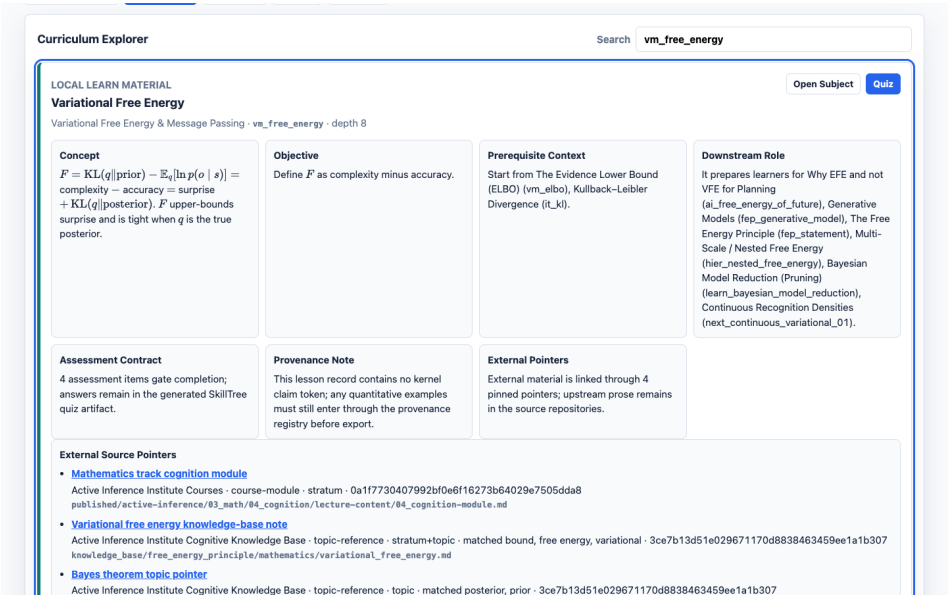


Figure 3: The Curriculum tab captured with the generated lesson record for the variational free energy skill opened through its Learn action. The panel shows the concept context, prerequisite position, downstream role, assessment summary, and the commit-pinned external source pointers generated for that skill, alongside the handoff buttons that return the learner to the official SkillTree subject and quiz routes. It is the delivery counterpart to the course-material provenance rule: local lessons link out to pinned upstream sources instead of copying them.

SkillTree export, learner-surface handoff, and generated front matter IV



Top subject-to-subject prerequisite flows

PREREQUISITE SUBJECT	DEPENDENT SUBJECT	DEPENDENCIES
Application Translation Practice `sub_practice_applications`	Open-Science Applications & Community Learning `sub_open_science_applications`	9
Markov Blankets & Self-Organisation `sub_blanket`	Open-Science FEP Interpretation `sub_open_science_fep`	8
Simulation, Provenance & Export `sub_comp_adv`	Open-Science Computational Reproducibility `sub_open_science_computation`	8
Expected Free Energy & Policy Selection `sub_efe`	Open-Science Active-Inference Studios `sub_open_science_active_inference`	8
Entropy & Divergence `sub_entropy`	Open-Science Information Literacy `sub_open_science_information`	8
Variational Free Energy & Message Passing `sub_free_energy`	Open-Science Variational Workflows `sub_open_science_variational`	8
Linear Algebra & Calculus `sub_linalg`	Open-Science Mathematical Learning `sub_open_science_math`	8
Probability Foundations `sub_prob_found`	Open-Science Probabilistic Learning `sub_open_science_probability`	8
Variational Free Energy & Message Passing `sub_free_energy`	Continuous Variational Inference `sub_continuous_variational`	7
Advanced Active Inference `sub_ai_adv`	Hierarchical Active-Inference Design `sub_hierarchical_active_inference`	6
Markov Blankets & Self-Organisation `sub_blanket`	Scientific Supplement: FEP Scope Tests `sub_scientific_supplement_fep`	6
Robotics, ML & Collective Behaviour `sub_broader`	Applied Evaluation & Source Dossiers `sub_applied_evaluation_dossiers`	6
Simulation, Provenance & Export `sub_comp_adv`	Scientific Supplement: Computational Traceability `sub_scientific_supplement_computation`	6
Expected Free Energy & Policy Selection `sub_efe`	Scientific Supplement: Active-Inference Terms `sub_scientific_supplement_active_inference`	6
Entropy & Divergence `sub_entropy`	Scientific Supplement: Information Geometry `sub_scientific_supplement_information`	6
Variational Free Energy & Message Passing `sub_free_energy`	Scientific Supplement: Variational Bounds `sub_scientific_supplement_variational`	6
Implementation, Simulation & Scaling `sub_implementation`	Reproducible Simulation Labs `sub_reproducible_simulation_labs`	6
Mutual Information & Bounds `sub_mutual_info`	Rate-Distortion & Decision Information `sub_rate_distortion_information`	6
Optimisation & Variational Calculus `sub_optimisation`	Continuous Dynamics & Stability `sub_continuous_dynamics`	6
Optimisation & Variational Calculus `sub_optimisation`	Scientific Supplement: Variational Mathematics `sub_scientific_supplement_math`	6

Figure 4: The accessible prerequisite-flow table in the Graph tab, scrolled past the subject lane canvas that sits above it in the same panel. Table and canvas are built from the same server-computed flow summaries, so the prerequisite structure reaches a keyboard or screen-reader user as captioned rows with real header cells rather than only as a drawing. Each row is one directed subject-to-subject relation: a prerequisite subject, the dependent subject that requires it, and how many skill dependencies run between them, ordered by weight.

SkillTree export, learner-surface handoff, and generated front matter V



```
      "stateDir": "/Users/4d/Documents/GitHub/projects/working/Active_Skillference/.active_skillference_local",
      "tokenEndpoint": "/api/skilltree/token"
    },
    "localDevelopment": {
      "defaultPassword": "",
      "defaultPasswordAvailable": false,
      "defaultUsername": "active-skillference-admin@localhost.invalid",
      "resetCommand": "uv run python scripts/reset_local_skilltree.py --confirm-local-reset",
      "showCredentialsCommand": "uv run python scripts/bootstrap_local_skilltree.py --show-local-credentials",
      "warning": "Local-development credentials only. Shared or deployed SkillTree instances must replace these defaults."
    },
    "statusUrl": "http://localhost:8080/public/status",
    "url": "http://localhost:8080"
  }
 },
 "skilltree": {
  "available": true,
  "body": {
    "clientLib": {
      "loggingEnabled": "false",
      "loggingLevel": "DEBUG"
    },
    "status": "OK"
  },
  "message": "SkillTree is reachable",
  "statusCode": 200,
  "url": "http://localhost:8080/public/status"
 },
 "liveSkilltree": {
  "checked": true,
  "complete": true,
  "expectedDependencies": 1199,
  "extraDependencies": 0,
  "extraDependencyExamples": [],
  "liveDependencies": 1199,
  "matchedDependencies": 1199,
  "missingDependencies": 0,
  "missingDependencyExamples": [],
  "skills": 630,
  "subjects": 111
 },
 "skilltreeDeployment": {
  "modes": [
```


Figure 5: The browser-visible runtime payload in the Runtime tab, scrolled past the deployment-mode matrix to the local development block. The default password field is an empty string and its availability flag is false, so the page reports the local username and the recovery commands while withholding the credential itself; no client secret and no bearer token appear anywhere in the payload. The same view carries the SkillTree reachability block and the live dependency parity block reporting complete parity with no missing or extra edges, which is the auth-boundary evidence in the form the browser actually receives it.